

The exam is open books, notes, printed materials. You are allowed web access. Consulting/discussing with anyone which includes asking questions in any forum during the exam is strictly prohibited. *Unless specified explicitly, assume 64-bit X86 architecture with multiple CPUs.*
All the best!

1. Select the correct answer(s) for the following questions. Note that, you have to *write all correct choices* to get the credits (no partial marks). No explanation required.

 $2.5 \times 4 = 10$

- (a) Consider two threads T_1 and T_2 of a process P . Which of the following statements are *true*.
- A. T_1 can access a pointer declared in the global scope after it is initialized by T_2
 - B. T_1 can access an address after it is allocated using `mmap` by T_2
 - C. T_1 can access a variable after it is allocated in the stack area of P
 - D. T_1 can access a variable after it is allocated in the stack area of T_2
 - E. T_1 can read from `abc.txt` using `fd` (declared as a global variable) after T_2 successfully executes `fd = open("abc.txt", O_RDWR)`.
- (b) Which of the following C/assembly statement(s) *may* result in a user-to-kernel mode switch? Assume that all addresses are virtual addresses and RAX and RCX are general purpose registers.
- A. `pop RAX; // Store the value at current SP into RAX`
 - B. `MOV RAX, RCX; // Move the value in register RAX to RBX`
 - C. `inc (RAX); // increment the value at memory address in RAX`
 - D. `add RAX, RCX; // RAX = RAX + RCX`
 - E. `ret; // Return from the current function`
- (c) Which of the following statement(s) is/are true regarding the page fault handler behavior where the page fault handler returns without resulting in any kind of fault and no signals like segmentation fault are raised to the faulting process? Note that, page fault handler can be invoked because of many reasons as discussed in the lectures. Assume a 4-level page table and 48-bit virtual address with a page size of 4KB.
- A. *At least one level* of the page table is required to be updated
 - B. *Exactly one level* of the page table is required to be updated
 - C. *At most four levels* of the page table is required to be updated
 - D. *At least one TLB entry* is flushed by the page fault handler
 - E. *At most one TLB entry* is flushed by the page fault handler
- (d) Consider the following pseudocode.

```
sem_t S1, S2;    /* S1 and S2 are initialized to zero */
string(){
    printf("A");
    wait(S1);
    wait(S2);
    printf("B");
    post(S1);
    printf("C");
    post(S2);
}

number(){
    printf("1");
    post(S1);
    printf("2");
    post(S2);
    wait(S1);
    wait(S2);
    printf("3");
}
```

Which of the following can be a correct output if one thread executes `string()` and the other thread executes `number()` concurrently?

- A. A12B3C
- B. A12BC3
- C. 123ABC
- D. 1A2BC3
- E. 12ABC3

2. True or False with justification. Justification should be a precise argument supporting your verdict. No marks without proper reasoning.

 $2 \times 5 = 10$

- (a) Assuming that the OS is part of the process address space (split-address mode), the OS can invoke a function in the user code segment (by using assembly instructions like `call` or `jmp`).
- (b) Allocation of kernel stack for a newly created process using `fork` can be delayed till it is scheduled for the first time.

- (c) Context switch overhead of shortest time to completion first (STCF) scheduling policy is always less than round robin (RR) scheduling policy.
- (d) If a process invokes `munmap(ptr, 8192)`, the handler of the system call *may* require flushing the TLB entries corresponding to the address range `ptr` to `ptr + 8192`.
- (e) In a file system employing write-back page caches (like linux page cache as explained in the lectures), synchronous and unbuffered write to a file (opened with `O_DIRECT` and `O_SYNC` flags) will not require any lookup of the file offset (to be written to) in the page cache.

The exam is open books, notes, printed materials. You are allowed to browse the web. Consulting/discussing with anyone which includes asking questions in any forum during the exam is strictly prohibited. *Unless specified explicitly, assume 64-bit X86 architecture with multiple CPUs.* Be precise with your answers. No marks for vague answers. Make reasonable assumptions and state them explicitly. *All the best!*

1. Consider the following program in the UNIX/Linux system.

```
#include<stdio.h>
#include<stdlib.h>
#include<unistd.h>
int main()
{
    int pid;
    int mpid = 0;
    for(int ctr=0; ctr<5; ++ctr){
        pid = getpid();
        if(pid % 2){
            mpid = fork();
        }
        if(mpid % 2 == 0)
            dup(1);
    }
}
```

Assume that all system call invocations are successful.

2 + 4 = 6

- (a) What is the maximum and minimum number of processes created during the execution of the above program (including the main process)?
 - (b) Let us define R to be the number of references to the `STDOUT` file object. Assuming the value of R is 1 at the start of the program (before execution of the `for` loop), what is the maximum and minimum value of R during the execution of the program?
2. In the classical `fork()` implementation, the child inherits the parent address space i.e., both the memory pages and page tables are copied. This requires creating duplicate content in memory at the time of `fork()` which can be inefficient, both in terms of memory and CPU usage, especially when the child process invokes `exec` immediately after returning from `fork`. We want to design a variant of `fork` (say `forkopt`) such that new executables can be launched faster than `fork` in a memory efficient manner.

10

While designing `forkopt`, you are not allowed to modify the semantic of `fork`. However, you may propose some restrictions related to the scheduling order (between the parent child process) and the nature of execution of the child process. Clearly explain the working of `forkopt` along with design and implementation aspects of `forkopt`. Further, if you are imposing any restrictions on the scheduling and/or on the nature of child process execution, describe why the restriction is required and what is your strategy to impose the restriction in the OS.

Hint: Maximum efficiency is achieved when the page table is shared between parent and child processes (similar to threads) as long as possible.

3. Memory is a precious resource and you want to augment the virtual memory subsystem of the OS to compress the used physical memory to increase the degree of multiprogramming without using swapping. Assume that there is an existing compression logic in the OS which provides functionalities like compression and decompression. Specifically, you can assume the following existing OS level functions,

14

`compressPFN(u32 srcpfn, void *outPTR)`: Compresses 4096 bytes of data in the page frame `srcpfn` and stores the compressed stream into `outPTR`. The length of compressed data is returned by this function.

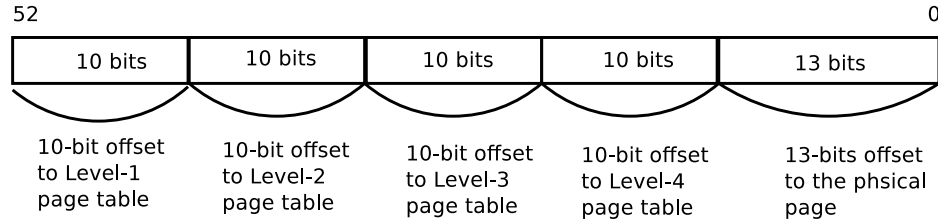
`decompressPFN(void *inPTR, u32 dstpfn, u32 size)`: Decompresses compressed data of length = `size` stored at `inPTR` and stores the decompressed data in the `dstpfn`. The caller of this function should ensure that the decompressed data is of length 4096 bytes.

The solution framework is as follows: There is a background user space process (a daemon, say `cmpserv`) which invokes a new system call i.e., `compress` in a periodic manner. The design of `compress` system call (parameters, return value and implementation) is to be explained as part of the answer. You are required to discuss the implementation of the `compress` system call—both its mechanism and policy.

Mechanism: How exactly the compress system call is implemented? What are the side effects on other OS subsystems (like virtual memory etc.) and your proposed methods of handling them? Note that, your implementation should handle both correctness and efficiency aspects.

Policy: In this part, you are required to discuss answers to questions like which memory pages to compress, how many memory pages to compress at what time etc. Note that, you should also discuss the resource tradeoff implications of your policy e.g., CPU usage vs. memory savings.

Hint: you may not want to compress all the memory pages because of performance reasons. Assume page size of 4KB and X86 like paging support.



4. Consider a 4-level paging system for fifty-three bit (53-bit) virtual address (as shown above) and a page size of eight kilobytes (8KB). For simplicity, assume that, the OS is executing on a separate address space and no OS address is mapped in the user process address space. Answer the following questions,

$$1 + 2 + 2 + 5 = 10$$

- If there are N processes, what is the maximum address space size of each process?
- What is the maximum amount of memory which can be allocated using a single page table page at every level till Level 3?
- The maximum physical memory supported by the above paging system is given as one petabyte (where 1 petabyte = 2^{50} bytes). If ten bits are used to maintain the access flags and five bits are reserved by the hardware, how many bits remain unused in any page table entry?
- If a process successfully unmaps 4GB memory from an existing mapping using a single `unmap` system call (every virtual address is uniquely mapped to a physical address), what is the maximum and minimum amount of free memory increase in the system? (ignore the memory usage change due to manipulation of VM related data structures like `vm_area` etc.)

5. Consider the following implementation as a solution to critical section problem for two threads T_0 and T_1 .

7

```
int flag[2] = {0,0};
int turn = 0;
void lock (int id) /*id = 0 or 1 */
{
    flag[id] = 1;
    while(flag[id ^ 1] && turn == (id ^ 1)); /* ^ is the bit-wise XOR*/
}
void unlock (int id)
{
    turn = id ^ 1; // (S1)
    flag[id] = 0; // (S2)
}
```

Does the above implementation guarantee mutual exclusion? If yes, provide a formal argument for correctness. If not, show a concrete example when the mutual exclusion is violated.

6. We would like to design a function that atomically dequeues the element E from the head of a queue $Q1$ and enqueues E at the tail of a queue $Q2$. The function `AtomicMove` shows a buggy attempt at designing such a function. Every queue object has a mutex M (initialized as `unlocked`).

5

```
void AtomicMove (Queue *Q1, Queue *Q2)
{
    mutex_lock(Q1->M);
    mutex_lock(Q2->M);
    Item E = Q1->Dequeue();
    Q2->Enqueue(E);
    mutex_unlock(Q2->M);
    mutex_unlock(Q1->M);
}
```

Point out the problem with this implementation and present a correct implementation.

7. Consider an indexed file system found in the UNIX operating system. The inodes have an indexed allocation scheme as follows: eight direct block address, four single-indirect block address and four double indirect block addresses. Assume that, the block size and on-disk inode size are four kilobytes (4KB) and 256 bytes, respectively. Block address in the system is 4 bytes long. Further, assume that every directory maintains an array of flat directory entries having the following structure,

```
struct dentry{
    u32 inode-number;
    char name[252];
}
```

Assume that the disk I/O operations are performed in units of block size i.e., 4KB. Answer the following questions.

$1 + 3 + 4 = 8$

- (a) You have written a program to create as many files as possible under an initially empty directory. How many files your program can successfully create?
- (b) A user process opens and successfully reads an existing file at `/home/user/courses/os/notes.txt` of size four kilobytes, where `/` is the file system mount point. Assume that, the super block, all the inodes and index meta-data required to access the file are present in memory and no other blocks are cached. What is the minimum and maximum number of disk block accesses required to complete the read operation?
- (c) In the above question, assume that the file `notes.txt` is of size 1GB and the user process opens the file (with the path same as previous question), after that it reads one block (4KB) at a random block offset (offset is a multiple of 4KB) successfully. Further assume that, the super block, all inodes are always cached. However, the file indexes and directory content *may or may not* be cached. No file data is cached. What is the minimum and maximum number of disk block accesses required to complete the read operation?

Q1. Select the correct answer(s) for the following questions.
Note that, you have to write all correct choices. (2 x 3 = 6)

(i) Which of the following operation(s) can be allowed only in the OS (kernel) mode? Assume X86 system.

- A) Changing the IDT descriptors
- B) Changing the virtual-to-physical translation information
- C) Changing the stack pointer register (SP)
- D) Changing the instruction pointer (IP)
- E) Changing the current privilege level

(ii) The second argument for correct execution of a program with the 'read' system call can be

- A) an address from the program stack segment
- B) an address from the code segment
- C) an address from the program initialized data segment
- D) an address from the program uninitialized data segment
- E) an address from the dynamically allocated data segment

Q2. True or False with reasoning (1-2 sentences). No marks without proper justification. (2 x 2 = 4)

- (i) "mov RAX, (RBX)" // Move Reg RAX value to the memory address in RBX. The OS will not intervene during the execution of the above instruction.
- (ii) Allocation of kernel stack for a newly created process can be delayed till it is scheduled for the first time.

- Q1. Select the correct answer(s) for the following questions. (2 x 3)
- (i) Which of the following statement(s) is/are true regarding system call handler implementation in the Unix/Linux System?
- A) The OS may read the user stack region in the system call handler
 - B) The OS may write to the user stack region in the system call handler
 - C) The OS may not return to the instruction address just after the system call instruction
 - D) The OS may never return to the same process after the system call execution
 - E) The OS may deschedule the process
- (ii) Consider any process 'P' executes 'sleep(5)' where 'sleep' is a system call. Which of the following statement(s) is/are true?
- A) 'P' will be descheduled and put to the wait queue
 - B) When 'P' is context switched, the instruction pointer register will contain a user-mode address
 - C) When 'P' is context switched, the instruction pointer register will contain a kernel-mode address
 - D) When 'P' is context switched, the stack pointer register will contain a user-mode address
 - E) When 'P' is context switched, the stack pointer register will contain a kernel-mode address

Q2. True or False with justification. No marks without proper justification. (2 x 2)

- (i) The root user of a system can modify the exception handling behavior of the OS.
- (ii) If a program accidentally performs a divide-by-zero, the OS can fix the issue and continue executing the program.

participating members *

Q1. Consider the following program which is compiled in the Linux OS to create an executable file named 'mexec'.

```
1 #include<stdio.h>
2 #include<stdlib.h>
3 #include<unistd.h>
4 int sum = 0;
5 int func(void){
6     pid_t pid = fork();    //Assume success
7     if(pid == 0 && sum < 10){
8         sleep(1);
9         execl("./mexec", "mexec", NULL); //Assume success
10        return 10;
11    }
12    return 20;
13 }
14 int main(void){
15     while(sum < 10){
16         sum += func();
17     }
18     printf("Sum = %d\n", sum);
19 }
```

Consider that we execute './mexec' on the bash shell. Answer the following questions,

- (a) What will the behavior of this program and why? (Give reasons using 2/3 sentences)
- (b) What is the maximum and minimum number of processes active at any point of time during the program execution (assume execution of the parents finish before their children invoke the 'execl' system call at line #9)?
- (c) How many PCBs will be created and destroyed during the execution of the program?
- (d) What will be the output of the program if the 'execl' at line #9 is commented?

(4 + 2 + 2 + 2 = 10)

CS330 - Quiz 1

Total: 15 marks

Duration: 20 mins

open books and notes, no mobile phones, friends disconnected during exam

Be precise, no marks for vague answers

Roll No:

Q1. Select the correct answer(s) for the following questions. No partial marks. (2×3)

- (i) You have added code at the beginning of the functions mentioned in the question to print the stack pointer every time the function is executed. Which of the following statement(s) are true regarding the value of stack pointer?
- (a) For a given user-level function, value of stack pointer *can be* different.
 - (b) For a given OS-level function which is exclusively used by a system call handler (no other OS code calls this function), value of stack pointer *can be* different.
 - (c) For two different OS-level functions, the value of stack pointer *can be* same.
 - (d) For a given OS-level function handling a particular interrupt, value of stack pointer *must be* same.
 - (e) For two different functions from two different application, value of stack pointer *can be* same.

Ans: a, b, c, e

- (ii) In a uniprocessor system, in which of the following case(s), a process executing in user mode can cause an entry into the OS?
- (a) accessing a general purpose register like **RAX**
 - (b) executing a **JMP** (jump) instruction
 - (c) decrementing an unsigned integer value stored in a register beyond zero
 - (d) executing a **printf()** statement
 - (e) returning from a function

Ans: b, d, e

- (iii) Which of the following statement(s) are true in a system with only segmentation (no paging)?
- (a) Two segment registers may point to the same segment descriptor
 - (b) If an ISA supports one segment register, run-time binding is not possible
 - (c) If an ISA supports one segment register, intra-application isolation is not possible
 - (d) If an ISA supports one segment register, isolation across applications is possible but OS memory can not be isolated
 - (e) If an ISA supports one segment register, inter-application isolation and OS isolation can be achieved

Ans: a, c, e

Q2. True or False with justification. No marks without proper reasoning. (2 × 3)

- (i) In a machine with a single CPU, an application executing an infinite loop will freeze the system.

Ans: *False*, The OS will get control of CPU resource to handle interrupts, especially the timer interrupts. The interrupt handler can switch out or kill the process.

- (ii) Without compromising isolation, two applications can communicate among themselves without the OS support.

Ans: *False*, when two processes are sharing a part of resource, to achieve control on the extent of sharing and check access to unshared part of the resource, the OS support is required.

True (less practical), in a scenario when the entire resource is only used by the two application and no other application is using the resource.

- (iii) In a hierarchical paging system, for any given virtual address, page table entries at each level must have same access permissions (read, write and execute).

Ans: *False*, page table entry at upper level dictates the permission for a larger VA space which might require different permissions. For example, a level-1 entry in 48-bit address covers 512 GB memory range which can be shared by code and data, and require different permissions.

Q3.

```
void initialize_array(void)
{
    int array[4];
    int ctr=4;
    do{
        ctr--;
        array[ctr] = 0;
    }while(ctr >= 0);
    printf("%d\n", array[3]);
}
```

What will be the output of the above function in X86 systems? Explain. (1×3)

Ans: The program will execute infinitely and will not terminate.

This behavior is primarily due to two reasons,

- The loop will terminate when the value of $ctr = -1$, which implies the program will access $array[-1]$.
- $\&array[-1] = \&ctr$: Since the stack grows from higher address to lower address

CS330 - Quiz 2

Total: 15 marks

Duration: 20 mins

open books and notes, no mobile phones, friends disconnected during exam

Be precise, no marks for vague answers

Roll No:

Q1. Select the correct answer(s) for the following questions. No partial marks. (2×4)

(i) Which of the following statement(s) are true in virtual memory system with paging?

- (a) Address of a variable in a program, if printed every time the program executes, *can be* same.
- (b) Address of a variable in a program, if printed every time the program executes, *can be* different.
- (c) Address of a variable *can be* changed by the OS during program execution.
- (d) Address of a library function (e.g., printf), if printed by two applications, *can be* same.
- (e) Address of a library function (e.g., printf), if printed by two applications, *can be* different.

Ans: a, b, d, e

(ii) In which of the following scenario(s), a page fault can occur?

- (a) accessing a general purpose register like **RAX**
- (b) executing a JMP (jump) instruction
- (c) decrementing an unsigned integer value stored in a register beyond zero
- (d) returning from a function
- (e) writing to an allocated and accessed variable

Ans: b, d, e

(iii) Which of the following statement(s) are true?

- (a) A superuser (e.g., root user in Unix) can execute all privileged instructions.
- (b) If OS accesses a virtual address which is not mapped in the OS page tables, system will crash.
- (c) If OS accesses a virtual address which is not mapped in the OS page tables, a page fault will occur.
- (d) The OS page fault handler *may* terminate a process.
- (e) The OS page fault handler *must* invoke swapping.

Ans: c, d

(iv) Which of the following statement(s) are true?

- (a) For a TLB without ASID support, a full TLB flush is *always* required when switching between two applications
- (b) For TLB without ASID support, a full TLB flush is *always* required on user to OS switch
- (c) If the OS updates a page table entry, it must flush the corresponding TLB entry from the TLB (with ASID support)

- (d) If the OS updates a page table entry, it must flush the corresponding TLB entry from the TLB (without ASID support)
- (e) If OS swaps out a page, it must flush the corresponding TLB entry

Ans: a, c, d, e

Q2. Consider a paging system with 48-bit virtual address, 4-level page tables and page size of 4KB. At each level, 9-bits of virtual address are used to index into the page table entry location. An application allocates total 1GB of virtual address by allocating 4K aligned random and distinct virtual pages. Considering all the virtual addresses are mapped to physical addresses, calculate the best case and worst case memory usage (in KB) to maintain the page tables for this application. (1×7)

Ans:

L4 (9bits), L3 (9bits), L2 (9bits), L1 (9bits), page offset (12bits)

Virtual address size = 1GB = 2^{18} pages

$\Rightarrow$ # of entries required at L1 = 2^{18}

Best case (3 Marks)

L1: 2^{18} entries require 2^9 page table pages

L2: 2^9 entries require 1 page table page

L3: 1 entry require 1 page table page

L4: 1 entry require 1 page table page

Total page table memory = $(2^9 + 1 + 1 + 1) \times 4 \text{ KB}$

Worst case (4 Marks)

L1: 2^{18} entries can be spread across 2^{18} different page table pages (max. page table pages = 2^{27})

L2: 2^{18} entries can be spread across 2^{18} different page table pages (max. page table pages = 2^{18})

L3: 2^{18} entries can be spread across 2^9 different page table pages (max. page table pages = 2^9)

L4: 2^9 entries require 1 page table page (max. page table page = 1)

Total page table memory = $(2^{18} + 2^{18} + 2^9 + 1) \times 4 \text{ KB}$

Total: 15 marks

Duration: 25 mins

*open books and notes, no mobile phones, friends disconnected during exam***Be precise, no marks for vague answers**

Roll No:

Q1. Select the correct answer(s) for the following questions. No partial marks. (2×4)

- (i) In a uni-processor system, which of the following statement(s) are true regarding threads of a multi-threaded process?
- (a) More than one thread can be inside system call handler only if they have separate OS stacks.
 - (b) More than one thread can be in RUNNING state if they have separate OS stacks.
 - (c) More than one thread can be inside system call handler if interrupts are disabled during system calls.
 - (d) More than one thread can be inside system call handler if context switching is disabled during system calls.
 - (e) None of the above.

Ans: a,c

- (ii) Which of the following statement(s) are true?

- (a) If a process is interrupted during system call handling, the OS will crash.
- (b) If a process is de-scheduled during system call handling, it must resume its execution in OS mode.
- (c) If a process is waiting for an I/O event, it must resume its execution in OS mode.
- (d) A multitasking OS can disable interrupts whenever the CPU is executing in OS mode.
- (e) None of the above.

Ans: c

- (iii) Which of the following statement(s) are true?

- (a) Multi-level feedback queue scheduling favors I/O bound processes.
- (b) Starvation problem in multi-level feedback queue scheduling can be addressed by carefully tweaking the value of time quantum.
- (c) Dynamically determining the value of time quantum can address the issue of a malicious user tricking the multi-level feedback queue scheduler.
- (d) Standard deviation of waiting time can be used to measure the fairness of any scheduling algorithm.
- (e) For a given scheduling algorithm, average waiting time monotonically increases with number of processes.

Ans: a, b, c, d

- (iv) Which of the following statement(s) are true?

- (a) There can be a common signal handler function for multiple signals.
- (b) If two processes execute `kill(pid, SIGSEGV)` around the same time, one of the calls may fail.
- (c) If two processes execute `kill(pid, SIGSEGV)` around the same time, the target process (with `PID=pid`) may execute the registered signal handler twice.
- (d) If two processes execute `kill(pid, SIGSEGV)` around the same time, the target process (with `PID=pid`) may execute the registered signal handler only once.
- (e) Within a signal handler function, a signal can be generated to another processes using the `kill` system call.

Ans: a, c, d, e

Q2. Consider the following C program. (4)

```
1 #include<stdio.h>
2 #include<stdlib.h>
3 #include<signal.h>
4 char *ptr1 = NULL;
5 char *ptr2 = NULL;
6 void sighandler(int signo)
7 {
8     printf("I am in sighandler\n");
9     ptr1 = ptr2;
```

```

10 }
11
12 main()
13 {
14     if(signal(SIGSEGV, &sighandler) < 0)
15         perror("signal");
16
17     ptr2 = (char *)malloc(100);
18     ptr2[0] = 'A';
19     printf("%c\n", *ptr1);
20 }

```

What will be the output of this program when executed in a UNIX/Linux OS? Explain.

Ans: The program will print `I am in sighandler` infinitely.

Reason: The program counter (RIP) will point to the instruction that dereferences memory location pointed to by `ptr1`. The memory address `ptr1` is stored in a register (e.g., RAX) before it can be dereferenced. Note that, you have seen this behavior in Assignment-II for page fault handler implementation where if the register state is not saved and restored, the accessed memory location changed when the instruction gets re-executed after exception handling. Therefore, after the page fault and `sighandler` execution, the value of the register involved remain unchanged and causes repeated SEGFAULTs.

Q3. Consider the following C program. (3)

```

1 #include<stdio.h>
2 #include<stdlib.h>
3 #include<unistd.h>
4 #include<fcntl.h>
5 main()
6 {
7     char buf[8];
8     int fd, dfd;
9     fd = open("some.txt", O_RDWR);
10    if(fd < 0){
11        perror("open");
12        exit(-1);
13    }
14    dfd = dup(fd);
15    buf[7] = 0;
16    read(fd, buf, 7);
17    printf("%s\n", buf);
18    read(dfd, buf, 7);
19    printf("%s\n", buf);
20 }
~

```

Assume that `some.txt` is a valid file whose content is: `AAAAAABBBBBBBBCCCCCCC...` What will be the output of the above program when executed in the UNIX OS? (no explanation needed)

Ans:

AAAAAAA
BBBBBBB

The `dup` system call simply create a duplicate file descriptor, the file object pointed to by the two file descriptors remains the same. (See slide-17, IPC.pdf)

Total: 15 marks

Duration: 25 mins

*open books and notes, no mobile phones, friends disconnected during exam***Be precise, no marks for vague answers. Make reasonable assumptions.**

Roll No:

Q1. Select the correct answer(s) for the following questions. No partial marks. (2×4)

- (i) Block usage bitmap structure is used to keep track of free blocks in the storage device. Assume that, if the i^{th} bit in block bitmap is 1, then the i^{th} block is in use. If a file system employs block bitmaps, for which of the following block usage scenarios, the corresponding bit in the block bitmap *must be* one?
- (a) The block is used to store any file inode.
 - (b) The block is used to store any directory inode.
 - (c) The block is used to store file content.
 - (d) The block is used to store contents of a directory.
 - (e) The block is used to store inode bitmap.

Ans: c, d

- (ii) If a file is read using `read(buf, K)` system call where K is the number of blocks to be read, the total number of disk block reads required to serve the request
- (a) is equal to K .
 - (b) is at least K .
 - (c) is at most K .
 - (d) can be more than K .
 - (e) can be less than K .

Ans: d, e

- (iii) Consider a flat directory organization with variable sized entries spanning K disk blocks, directly addressable from the inode. Which of the following statements are true for any given directory? Assume that inode is present in memory and there is no caching at any layer.
- (a) To create a new file inside the directory, at least K block reads are required.
 - (b) To rename an existing file inside the directory, at least K block reads are required.
 - (c) To delete a file inside the directory, at least K block reads are required.
 - (d) To delete the directory, at least K block reads are required.
 - (e) To rename the directory, at least K block reads are required.

Ans: {a, b, d} or {a, b}

- (iv) Which of the following statement(s) are true regarding file systems organized into multiple groups (like ext4)?
- (a) If a file inode is created in a group, only the inode bitmap of that group is updated.
 - (b) If the size of a file in a particular group expands, only the block bitmap of that group is updated.
 - (c) The superblock may be repeated across all groups for increased reliability.
 - (d) If a file inode belongs to a particular group, then all the data blocks of the file must be from the same group.
 - (e) Number of blocks provisioned to store inodes is same for all the groups.

Ans: a, c, e

Q2. (2 + 1 + 2 + 2 = 7)

Consider an indexed file allocation found in the UNIX operating system. The file block index structures contained in the inode are as follows: nine direct block addresses, five single-indirect block addresses, and two double-indirect block addresses. The file system block size is 4KB and block address is 32-bits.

- (i) What is the maximum file size supported by this file system?
- (ii) What is the maximum disk size supported by this file system?
- (iii) What is the actual disk usage to store four files of 5GB each (ignore the space usage for storing inodes)?
- (iv) If a user requests to read a single block from a file of size 1GB at a random file offset (4KB aligned), calculate the minimum and maximum size of disk read required to serve the request? Assume that, the disk inode corresponding to the file is present in memory.

Ans:

(i) Number of entries in each indirect block = $4KB / 32bits = 1024$

Max file size = $(9(DI) + 5 \times 1024(SI) + 2 \times 1024 \times 1024(DI)) \times 4KB$

(ii) $2^{32} \times 2^{12} = 16TB$

(iii) File of size 5 GB requires 1310720 data blocks. $1310720 = 9$ data blocks from direct pointers + $5 * 1024$ data blocks from SI + X data blocks from DI

$\Rightarrow X = 1310720 - 5120 = 1305591$

$\Rightarrow \# \text{ of SI blocks} = \left\lceil \frac{1305591}{1024} \right\rceil = 1275$

$\Rightarrow \# \text{ of DI blocks} = \left\lceil \frac{1275}{1024} \right\rceil = 2$

$\# \text{ of data blocks} = 9 + 5120 + 1305591 = 1310720$

$\# \text{ of indirect blocks} = 5 + 2 + 1275 = 1282$

Total = $1310720 + 1282 = 1312002$ blocks

= 5248008 KB

For four files, disk usage = $4 * 5248008$ KB

(iv) If the file offset corresponds to any direct block, then one (1) disk read is required.

The last block of the file will require a double indirect block access. Therefore, maximum three (3) disk reads are required.

Total: 15 marks

Duration: 25 mins

*open books and notes, no mobile phones, friends disconnected during exam***Be precise, no marks for vague answers. Make reasonable assumptions.**

Roll No:

Q1. Select the correct answer(s) for the following questions. No partial marks. (2×4)

- (i) Consider a file system using a block layer disk cache with delayed write semantics to store disk blocks in memory to improve application performance. Assume that M_B represents the memory page containing the contents of block B . Given that the file system provides APIs to perform both direct I/O and cached I/O, which of the following statement(s) are true?
- (a) M_B must be written back to the disk when the cached block is evicted.
 - (b) M_B must be written back to disk if it is dirty (modified in memory).
 - (c) In some cases, M_B is required to be written back to the disk even when it is not modified.
 - (d) M_B is written back to the disk if and only if it is modified.
 - (e) None of the above.

Ans: e

- (ii) In addition to information provided in (i), consider that B is mapped to K^{th} block of a file F (regular file with no links). Which of the following statement(s) are true?
- (a) M_B can be in memory even before K^{th} file block offset is read.
 - (b) M_B can be in memory even before K^{th} file block offset is read for the first time after mounting the FS.
 - (c) M_B can be in memory even when the file is closed.
 - (d) M_B can be in memory even when the file is renamed.
 - (e) M_B can be in memory even after many write operations on F .

Ans: {a, b, c, d, e}

- (iii) Execution of *fsck* utility for a given file system before mount revealed a mismatch between the number of data blocks calculated from the data block bitmap and all inode data pointers. The possible reason(s) *can be* a FS crash
- (a) during creation of an empty file.
 - (b) during creation of an empty directory.
 - (c) during truncation of a file.
 - (d) during deletion of a file or directory.
 - (e) during unmount of the file system.

Ans: {a, b, c, d, e}

- (iv) Which of the following statements are true to support a multi-threaded process on a multi-processor system? (Assume gemOS like design on a multi-processor system)
- (a) Concurrent execution of `expand()` system call *must not* be allowed.
 - (b) The `expand()` system call handler is required to use locks.
 - (c) The `expand()` system call handler is required to use locks and disable interrupts.
 - (d) Page fault exception handler must use locks.
 - (e) Page fault exception handler must use locks and disable interrupts.

Ans: {b, d}Q2. $2 \times 3.5 = 7$

Consider the following code segment executed from two threads.

```
static int var = 0, ctr = 0;
func()
{
    for(ctr=0; ctr<1000; ctr++){
        var++;                // Translates to a single instruction INC
                             // Increments var using three implicit micro-operations
    }
}
```

Assume that, the compiler translates the C statement `var++` to a single instruction—`INC (&var)`. If the code is executed concurrently by two threads to completion, what is the possible minimum and maximum value of `var` in a uniprocessor system? What will be the output in a multiprocessor system?

Uniprocessor system

Minimum: **1000**. Explanation: If both threads set `ctr` to zero in the beginning and before one checks the condition, the other loops 1000 times, setting `var` and `ctr` to 1000 and quit. The first thread's condition will fail and quit. Given that for any thread, `ctr++` happens only after the corresponding `var++`, the 1000 increments to `var` is inevitable.

Maximum: There are two possible answers assuming the behavior of `ctr++` followed by `ctr<1000`.

Steps: { 1) *mov (ctr), R*; 2) *add 1, R*; 3) *mov R, (ctr)*; 4) *cmp R, 1000*}

Ans = **501500**. Explanation: Assume that T2 executes the loop 1000 times and gets descheduled before updating `ctr` and performing the loop condition check. T1 is scheduled and executes the loop only once and gets descheduled before executing the third instruction. T2 resumes execution and reads the value of `ctr` (which is 0), updates it to 1 and iterates the loop 999 times before getting descheduled. This process continues.

$$var = 1000 + 1 + 999 + 1 + \dots + 1 + 1$$

$$= (1000 * 1003) / 2 = 501500$$

There is another possibility where the steps are as follows,

Steps: { 1) *mov (ctr), R*; 2) *add 1, R*; 3) *mov R, (ctr)*; 4) *cmp (ctr), 1000*}

In this case the answer will be: **502500**. You can verify this performing similar analysis

Multiprocessor system

Minimum = **1**. Explanation: T0 (on a very slow core) and T1 (on a high-speed core) read `count` into their register, T1 finishes all iterations and sets the value to 1000 (`ctr=1000`). T0 sets value to 1 and terminates.

Maximum is same as uniprocessor system.

Note that, answers with loop count = 100 will be accepted.

Quiz9

abhaymishra3420@gmail.com [Switch account](#)



The name and photo associated with your Google account will be recorded when you upload files and submit this form. Your email is not part of your response.

* Required

Group No# *

Your answer

Roll No of members participated (comma separated) *

Your answer

Answer all the questions in the provided text area.

Q1. Consider the following file related system calls executed successfully from the user space. For each system call, mention (with justification) if the system call can be completed without resulting in any disk I/O during the execution of the system call. Assume a file system with block-layer caching as discussed in the lectures. Your answer should be "Yes" or "No" followed by a brief justification for each of the following system calls. (2.5 x 4)

- (i) open
- (ii) open with O_CREAT flag
- (iii) write
- (iv) mount

Your answer



(OR) Submit a PDF/Photo of your answer mentioning the group no and roll no of participating members

 [Add file](#)

Submit

[Clear form](#)

Never submit passwords through Google Forms.

This content is neither created nor endorsed by Google. [Report Abuse](#) - [Terms of Service](#) - [Privacy Policy](#).

Google Forms

